Organize Your Code - Modular Programming

Using .c and .h files in C programming

In C programming, modular programming means splitting your code into smaller, reusable parts. This makes your code easier to read, maintain, and debug.

The most common way to organize C programs is by using separate **.c** files and **.h** header files.

Even though this approach is not required for small beginner programs, it is very useful for larger projects or more experienced programmers who want to keep their code clean and well-structured.

Why Use Header Files?

- To declare functions that are defined in another file
- To share variables, constants, or macros across files
- To organize your code into logical modules

Tip: Header files usually contain function declarations, macros, and struct definitions.

Example: Creating a Header File

In this example, you will learn how to create your own header file and use it to organize code across multiple files.

Let's create a simple **calculator module** with a header file and a source file.

1. calc.h

Start by creating a header file named **calc.h** to declare the functions:

```
#ifndef CALC_H
#define CALC_H

int add(int x, int y);
int subtract(int x, int y);

#endif
```

Example Explained

The **#ifndef**, **#define**, and **#endif** lines are called an **include guard**.

They prevent the file from being included more than once by mistake, which can cause errors during compilation.

This is a common and recommended practice in all C header files.

This file itself contains declarations of two functions: **add()** and **subtract()**.

Writing the Function Definitions

2. calc.c

Next, write the function definitions inside your **calc.c** file:

```
#include "calc.h"

int add(int x, int y) {
  return x + y;
}

int subtract(int x, int y) {
  return x - y;
}
```

This file defines the functions declared in **calc.h**.

Using the Module in main.c

3. main.c

Finally, write the main program in **main.c** and include the header file to use the calculator functions:

```
#include <stdio.h>
#include "calc.h"

int main() {
  printf("5 + 5 = %d\n", add(5, 5));
  printf("6 - 4 = %d\n", subtract(6, 4));
  return 0;
}
```

Result:

```
5 + 5 = 10
6 - 4 = 2
```

How to Compile Multiple Files

When your program is split across multiple files, you need to compile them together. For example:

```
gcc main.c calc.c -o program
```

This tells the compiler to build one executable named **program** from both **main.c** and **calc.c**.